Breaking Time: Methods, Artifacts, and Forensic Detection of Timestomping on FAT32, Ext3, and Ext4 File Systems

Author: Allan Korol, allan.korol@gmail.com
Advisor: *Domenica Lee Crognale*

Accepted: *2025-07-30*

Abstract

Hiding malicious files is imperative to breach a computer system successfully. To conceal malicious files among legitimate ones and complicate forensic investigations, adversaries often employ timestomping, which is the manipulation of file timestamps, as a defense evasion technique. This paper explores the diverse methods used to timestomp files on FAT, Ext3, and Ext4 file systems, focusing on how adversaries adapt their approaches based on available system access and permissions. The direct and indirect forensic artifacts left behind by these methods are analyzed, providing a framework to help investigators identify likely timestomping techniques. By correlating artifacts with specific adversary capabilities, this research supports faster and more accurate detection, enhancing incident response efforts on Linux systems.

1. Introduction

Timestamps are a critical pillar in Digital Forensics and Incident Response (DFIR) that enables analysts to reconstruct the sequence and scope of suspicious and potentially malicious activity (Mohamed & Khalid, 2019; Singh & Gupta, 2019). Within operating system internals, a complex chain of operations occurs when a program interacts with the file system to create, modify, or delete files. This includes the interaction between the program, standard libraries, and the kernel, affecting the file system and its metadata, such as timestamps (Thierry & Müller, 2022).

While timestamps are central to investigations, adversaries often manipulate them to obfuscate their actions and evade detection through various tools and techniques (Singh & Gupta, 2019; Yaacoub et al., 2021). One of the core tactics within the MITRE ATT&CK framework (ATT&CK, 2021), which has evolved into a cornerstone for mapping adversarial techniques and tactics, is Defense Evasion, which encompasses a range of techniques that adversaries employ to evade detection on compromised systems. Timestomping (T1070.006) is a form of anti-forensics that manipulates file timestamps to conceal malicious artifacts.

When considering techniques available to adversaries, the use of binaries that are a subset of known and trusted operating system binaries—Living off the Land Binaries (LOLBins)—is highly desirable as a method of evading detection (Chunduri et al., 2024). Within the Linux operating system, these system binaries allow for the manipulation of file timestamps, including the `touch`, `debugfs`, and `cp` (copy) binaries. These can be used to modify file timestamps, including access, modification, change, and creation times on various file systems, including FAT32, Ext3, and Ext4. Adopting a principle from a French criminologist, Edmond Locard, the Locard's Exchange Principle is commonly quoted as "every contact leaves a trace." Actions involving timestomping may include artifacts directly related to the file system or artifacts created as a result of the action in the operating system—indirect artifacts. This research is based on the assumptions of Locard's Exchange Principle. It examines the artifacts left behind by LOLBins that are used to manipulate file timestamps to provide a comprehensive

Author Name, email@address

understanding of the methods and artifacts available to adversaries, as well as methods of detection and mitigation.

Commented [JL1]: This should be upfront.

Commented [JL2R1]: Change this to further guidance on special formatting (code, file names, etc.)

2. Literature Review

Within the field of digital forensics, there has been a long-standing interest in the techniques used by adversaries to manipulate file timestamps. While not the earliest, Olsson and Boldt (2009) introduce research for the visualization of timestamps on investigations and the impact of timestamp manipulation (timestomping) on piecing together an accurate timeline of events.

In recent years, timestomping has become a recognized technique leveraged by adversaries to evade detection, classified as an Indicator Removal technique under the Defense Evasion tactic of the MITRE ATT&CK framework (ATT&CK, 2021). Within this framework, timestomping is described as a "technique that modifies the timestamps of a file (…) often to mimic files that are in the same folder and blend malicious files with legitimate ones" (ATT&CK, 2021). The MITRE ATT&CK framework is a cybersecurity ontology that standardizes terminology and enhances understanding of adversarial techniques across the community. While the framework may include the names of tools used by adversaries—the procedures defined within it—these are presented at a very high level and do not include detailed commands or artifacts that may result from using these tools. MITRE ATT&CK does provide detections for techniques; however, these are again at a high level and only provide a general indication of what data could be used to detect an adversary that may use a particular technique (Strom et al., 2018).

Discussing the challenges facing forensic practitioners when using tools, Bhat et al. (2021) outline the limitations of forensic tools in detecting timestomping and other anti-forensic techniques. The reliance on metadata containing timestamps, which adversaries can manipulate, is a key limitation of forensic tools that forensic practitioners must be aware of (Bhat et al., 2021). To that end, the research examines how

a selection of different forensic tools—The SleuthKit[1], Forensic Toolkit (FTK)[2], EnCase[3], and OSForensics[4]—handle various methods of data hiding, including timestomping. A key finding of the research is that forensic tools rely on the metadata of files and are, therefore, susceptible to inaccuracies when adversaries manipulate timestamps. This highlights the need for forensic practitioners to be aware of the limitations of forensic tools and that "all the components of timestamps should be performed to identify inconsistencies that may point toward forgery" (Bhat et al., 2021, p. 202).

When writing files to the file system, programs may intentionally use timestamps different from the time of the action. This pattern of behavior for LOLBins of the Linux operating system is explored in Forensic Focus (2015). The article examines how select programs and built-in operations—`wget`, `curl`, `scp`, copy (`cp`), and move (`mv`)—handle timestamps and update metadata, and provides a quick reference for investigators to understand what timestamps are expected when such tools are used. This also identifies nuances between tools that achieve the same goal, such as `wget` and `curl`, which are both used to download files but update the modified timestamps of a downloaded file differently. As with other research, this focuses on the impact on file system metadata and does not consider indirect artifacts that may be left behind in the operating system, such as logs or audit trails.

With the prevalence of anti-forensic techniques, Hosgor (2018) examines the methods of data hiding and artifact-wiping techniques that adversaries may employ. The outcome of the research is to provide recommendations and mitigations against various data-hiding techniques. Concerning timestomping, the research examined the impact of using the Metasploit Framework to delete timestamp metadata. While this showed the

---

[1] https://www.sleuthkit.org/

[2] https://www.exterro.com/forensic-toolkit

[3] https://www.opentext.com/products/encase-forensic

[4] https://www.osforensics.com/

tool was successful in deleting, the research focuses more broadly on data-hiding techniques and methods for detecting and mitigating these techniques. When examining the impact of Metasploit timestomping activity on the file and the deletion of the metadata, the research found that the dates had reset to epoch—Microsoft FileTime value of 1601-01-01 00:00:00–timestamps and that there were "big inconsistencies between the usual OS files and the manipulated ones" (Hosgor, 2018, p. 26).

With the emergence of machine-learning techniques in digital forensics, Mohamed and Khalid (2019) explore using machine learning to detect timestamp tampering. While this research does not explicitly focus on methods of timestomping, nor the artifacts left behind, it does provide for an approach to detect timestamp tampering relying solely on metadata of the filesystem—an element that has, in previous research, been identified as a limitation of forensic tools (Bhat et al., 2021). The method was applied to Microsoft NTFS file systems, but the research outcome was that the "build model was capable of predicting new tampered timestamps" (Mohamed & Khalid, 2019, p. 784). The value of the research of Mohamed and Khalid (2019) is that it provides a method of detecting possible timestomping activity, which could be used as an indicator for further investigation.

Pomeranz (2024) challenges the assumption regarding the ability to change certain timestamp artifacts of a file, in particular, birth and change times. The research examines the use of the `debugfs` command to manipulate file timestamps and the impact of this on the file system. This research provides a practical example of how timestomping of the birth and change time can be achieved using a LOLBin, but also highlights a limitation of using debugfs. Although `debugfs` is provided as a file system debugger[5], the command can be run on a mounted file system. However, in doing so, the file system cache in the kernel is not updated as the command operates on the raw file system, bypassing the kernel. This means that the file system metadata is not updated in memory, and therefore, the timestamps are not immediately reflected in the operating

[5] https://linux.die.net/man/8/debugfs

system. While the file system metadata is updated on disk, forensic tools examining the raw disk image will see the updated timestamps; the operating system will not reflect these changes until the cache is cleared or the file system is remounted, such as through a reboot. This key detail highlights the importance of performing live analysis during an investigation. Although this cache of filesystem metadata is not updated automatically, Pomeranz (2024) demonstrates how the `/proc` file system can be used to clear the cache, allowing the updated timestamps to be reflected in the operating system.

Existing research into timestomping techniques within the field of digital forensics has primarily focused on the methods used to manipulate timestamps rather than the artifacts produced by these actions. While understanding the techniques is important, the impact of timestomping can extend beyond the file system itself. Research has shown that the file system may not always be trustworthy (Bhat et al., 2021), and therefore, artifacts from the operating system may be required to prosecute a case. There is, therefore, a need for a more comprehensive understanding of the artifacts produced by timestomping techniques, including direct artifacts within the file system and indirect artifacts at the system level.

This research examines the methods used to timestomp files and the artifacts produced by these actions, both direct and indirect artifacts. This will allow investigators to better identify and respond to potential adversarial activity and ultimately improve the overall effectiveness of digital forensics investigations.

3. Research Method

This research is centered around designing and developing a test environment that isolates timestamp changes to facilitate the analysis of artifacts relevant to identified timestomping methods. This research examines the direct file system artifacts and indirect operating system artifacts left behind by various timestomping methods on FAT32, Ext3, and Ext4 file systems. The methods of timestomping that will be examined include:

- The `touch` command can change a file's access and modification times.

- The `cp` command, in general, is used to copy files and directories. There are, however, optional arguments, such as `--preserve`, that can be used to preserve the original files' attributes, including some timestamps.
- `debugfs` is a tool that can modify the file system directly. This tool is used during the environment to configure a file for the `cp` preserve strategy and has proven effective.
- System time manipulation (time travel) is a method that can change the system time, affecting the timestamps of files. This is achieved using the `date` command to set the system time to a specific value.

3.1. Testing Environment and Design

To investigate the forensic artifacts left behind by various timestomping techniques, a controlled testing environment will be established using multiple file system types commonly found on Linux systems: FAT32, Ext3, and Ext4. The procedure for setting up the test environment, executing the timestomping methods, and examining direct and indirect artifacts is detailed in Table 3, included in the Appendix.

The testing environment is constructed through four stages. The first three stages build to the point of producing a test image that can be used to conduct the experiments.

The fourth stage is the creation of each virtual machine for the testing of the timestomp strategy and analysis of artifacts left behind. The first three stages are as follows:

1. **Creation of base Operating System (OS) image:** A minimal installation of Ubuntu 24.02 was created in a virtual machine with the specifications outlined in Table 1. This base image of the operating system is copied and used for the setup process of creating the file systems, and another copy is used to create a test image for testing. For each timestomp strategy used, a new copy of the test image is created to ensure that the file system is in a known state, and a subsequent copy of the disks contains only those changes related to the specific timestomping strategy being tested. A

further disk image was added to be used and modified by strategies during the setup process. This file system emerged because of the need for a file system that could be pre-stomped via the `debugfs` command and would not interfere with the file system images or the operating system image. The primary use case for this pre-stomped file system was the `cp --preserve` command that needed a file with the specific time attributed prior to execution.

2. **Creation of file systems:** Using a copy of the base image, the FAT32, Ext3, and Ext4 file systems are created using a series of disk and file system initialization commands, including `fdisk` and the `mkfs` command. The file systems are also populated with a control file with a known, specific name and a series of other files and directories.
3. **Creation of test image:** Once the file systems are created—including the stomped file system—the files are copied to the host machine and converted to a VDI file format using the VBoxManage `convertfromraw` command. These VDI files are attached to the virtual machine. Once the Virtual machine is started, the file systems have persistent mounts in the operating system through entries in the `/etc/fstab` file. At this point, any additional setup methods for each strategy are executed. This is achieved with a visitor software design pattern (Gamma et al., 1995) for each timestomp strategy.

Aspect	Specification
Virtualization	VirtualBox 7.1.8 r168469
Operating System	Ubuntu 24.02
CPU	1 vCPUs Intel(R) Core(TM) i7-14700 2.10 GHz
RAM	1 GB

Disk Space	8 GB
Network	NAT
File System	Ext4

Table 1: Hardware and Operating System specifications of Virtual Machines provisioned for testing

4. **Creation of execution run image:** The test image is created by copying the test image with all the attached file systems to the virtual machine. Once the virtual machine is running, the timestomp strategy is executed via one or more SSH commands, as determined by the tested timestomp strategy. The virtual machine is powered off, and a copy of the test image is created for further analysis.

The relationships between the different virtual machines are shown in Figure 6, included in the Appendix. The base image is used in both the setup VM and test VM, with the test VM then used to create the execution run image, which is used to conduct the experiments.

Each data file system will be created and mounted into a Linux virtual machine. Timestomping will be conducted using a range of techniques. Following each timestomping event, both direct (file system-level) and indirect (system-level) forensic artifacts will be examined to evaluate the presence and persistence of evidence left by each method.

By using isolated file systems within virtual machines, this approach enables realistic testing on a live operating system while containing and observing filesystem-level behaviors. This setup supports the identification of both low-level and high-level indicators of timestomping activity. Virtual environments also allow for controlled, repeatable experiments under varying configurations, ensuring consistency and reliability in the results. This approach ensures that the dependent variables—the direct and indirect artifacts left behind by the timestomping methods—can be isolated from the independent variable—the tested timestomping methods.

Author Name, email@address

3.2. Analysis

Analysis of preserved operating system states is performed using automated and manual methods. Automated methods comprise live analysis, built into the testing framework—which allows the examination of the file system as the operating system sees it immediately following the timestomping event—and capabilities that use the disk image inspection library DiscUtils to determine the overall success of the timestomping action on the raw data on the disk. The distinction between what the operating system sees and what is on the disk is an important consideration prevalent in Pomeranz (2024), which highlights the need to flush the operating system cache to ensure that the updated file system metadata is loaded.

Manual methods involve the analysis of disk hex-dumps—as a source of indicators of potential timestomping activity—and examining the raw disk images in the forensic image mounting tool *FTK Imager*. For Ext-based file systems that include a journal, an analysis of the filesystem journal will also be performed to identify any events that may indicate timestomping activity.

3.3. Timestomp Methods Investigated

The testing environment will investigate some of the most common timestomping methods used by adversaries. The methods that will be investigated include:

- **Touch:** The Linux `touch` command can change a file's access and modification times. The change time is the last time the metadata structure was modified. This is important as it can be used to identify when a file was last accessed or modified, which can be helpful in forensic investigations.
- **cp:** The Linux cp command can copy files and directories. The –preserve option can preserve the original files' attributes, including timestamps. This is useful for timestomping as it allows the adversary to copy a file with the original timestamps intact, making it harder to detect the timestomping activity.

- **debugfs:** `debugfs` is a tool designed to diagnose and repair Ext-based filesystem problems and modify the file system directly. As this tool effectively modifies file system metadata, it is leveraged in this research to modify one of the file systems used in the testing environment, where precise control over the timestamps is required. Anecdotal research via an attempt to run `debugfs` on a mounted file system indicates that the time does update; however, a copy of the file metadata in memory is not updated. Pomeranz (2024) details this activity and the need for subsequent commands to update the copy of the metadata stored in the kernel.
- **Time-travel:** Time-travel is a method of manipulating the system time to change the timestamps of files. This is achieved using the date command to set the system time to a specific value. Once the system time has been set to a date in the past, any actions will take the time as the current time (in the past), and therefore, any files created or modified will have the timestamps set to that date. By combining this action with a one-liner command, the impact on indirect artifacts may be minimized, which is a key focus of this research.

4. Results

The results are presented in two sections: *direct artifacts*, which are those that are directly related to the file system, and *indirect artifacts*, which are those that are associated with the operating system.

4.1. Direct Artifacts

Examining direct artifacts focuses on the filesystem-level changes due to the timestomping methods. Analysis of these artifacts required a strong background in disk structures—which are used to organize the disk and identify the location of partitions and files—and file system structures, which store files and directories on the disk.

4.1.1. FAT32

For the analysis of FAT32 file systems, the Linux Kernel Documentation for VFAT file systems was used to understand the file system structures and how they are used to store files and directories. This was then examined by mounting the file system image into FTK Imager and analyzing the file system structures and the timestamps of the files. A summary of the results of the timestomping methods on the FAT32 file system is shown in Table 2.

The VFAT file system documentation provided the structure of a FAT32 file system record, detailed in Listing 1, with the size of a `char` being a single byte. Within this structure, the `ctime`, `cdate`, `adate`, `time`, and `date` fields store the creation time, creation date, access date, modification time, and modification date of the file, respectively, and are the focus of the analysis.

```
struct directory {
      // Short 8.3 names
      unsigned char name[8]; // file name
      unsigned char ext[3]; // file extension
      unsigned char attr; // attribute byte
      unsigned char lcase; // Case for base and extension
      unsigned char ctime_ms; // Creation time, milliseconds
      unsigned char ctime[2]; // Creation time
      unsigned char cdate[2]; // Creation date
      unsigned char adate[2]; // Last access date
      unsigned char reserved[2]; // reserved values (ignored)
      unsigned char time[2]; // time stamp
      unsigned char date[2]; // date stamp
      unsigned char start[2]; // starting cluster number
      unsigned char size[4]; // size of the file
};
```

Listing 1: FAT32 File System Record Structure

As an example of the analyzed VFAT records, Figure 1 shows the hex-dump result from FTK Imager of the first few hundred bytes of the partition. This has been

highlighted to identify the filename and the timestamps that are of interest. The VFAT record includes an extended header in the first 32 bytes. Within this, it is possible to see the UTF-16 name of the file, as compared to the 8.3 name—with only uppercase characters—in the first 8 bytes of the record from offset 0x20.

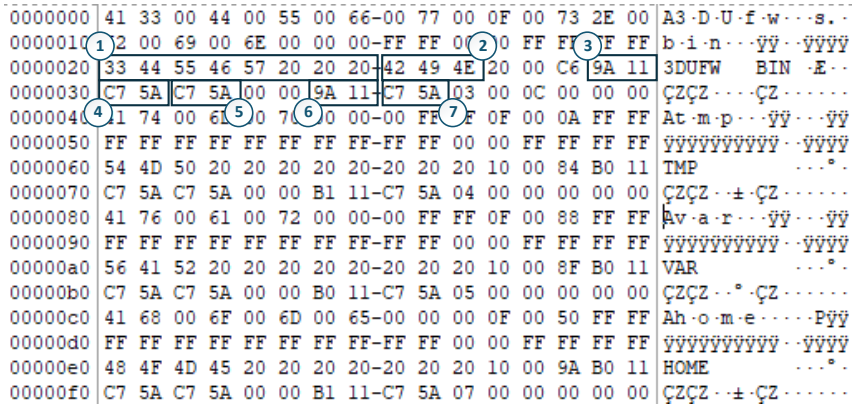


Figure 1: VFAT File System Record Hex-Dump with the following fields highlighted: 1. name, 2. extension, 3. `ctime` (creation time), 4. `cdate` (creation date), 5. `access date (adate)`, 6. `time` (modified time), and 7. `date` (modified date). The `reserved` field is evident by the 00 00 between the `adate` (5) and `time` (6) fields.

touch Command

Initial testing did not reveal any observable changes to the file system, which was hypothesized to result from file system caching. Checking the timestamp metadata of the file from the operating system via the `ls` or `stat` commands showed the expected changes, indicating the operating system was using the cached metadata. A subsequent execution, incorporating an explicit `umount` operation to ensure cache flushing, demonstrated that the expected changes were present.

Analyzing the raw data shows that the `touch` command changes the access and modified timestamp but does not change the creation timestamp of the file. Further, with no inbuilt tools to interact with the file system at a low level, unlike the debugfs

command for Ext3 and Ext4, the only option would be to write changes to the raw disk; calculate the offset of the file metadata, and write the changes directly. However, this is time-consuming and difficult, requiring root privilege. Even if this were to be achieved, the file system's cache, as held by the kernel, would not be updated. This means the changes are not reflected in the operating system until the file system is unmounted or the cache is cleared.

cp Command

The `cp` command was used with the `--preserve` option to copy the file from a file system with a pre-stomped file with the desired timestamps. The `cp` command was run with the `--preserve` option to ensure that the original timestamps were preserved as much as possible. The cp command experienced the same issue as the `touch` command, where the changes were not immediately reflected on the file system due to the file system cache. Again, this test was executed a second time with an umount command, which resulted in the changes as expected. It is noted that when the file existed, creation dates were not changed, and it appears that the contents were copied, and the access and modified timestamps were updated.

Time Travel

For time travel, research found that a series of commands can successfully change the date and time of the operating system, allowing for the change of the file's timestamps, including the creation time for direct artifacts. A review of the indirect artifacts that result from the time-travel operations and the impact on the Operating System is examined in 4.2. Indirect Artifacts. Therefore, looking at the direct artifacts only, the commands used to successfully change the timestamp, including creation, of the file are as follows:

```
setDate=$(date+”%Y-%m-%d%T”)
timedatectl set-ntp false
timedatectl set-time ”2008-10-0622:47:00”
mv /mnt/fat32/3DUfw.bin ./test.bin
mv ./test.bin /mnt/fat32/3DUfw.bin
```

```
touch /mnt/fat32/3DUfw.bin
timedatectl set-time $setDate
timedatectl set-ntp true
```

Listing 2: FAT32 File System Record Structure

These commands were combined into a single, one-liner command to minimize the potential impact on indirect artifacts. It should be noted that in this instance, the `ntp` service is disabled. It was found that the `timedatectl` command could not set the time when the `ntp` service was enabled.

	Target Timestamp: 2008-10-06 22:47:00 Encoded Date: 0x9A11 Encoded Time: 0xE0B5		
Method	**Created Timestamp**	**Accessed Date**	**Modified Timestamp**
`touch`	2025-06-07 02:12:52	2025-06-07	2025-06-07 02:12:52
`touch` with `umount`	2025-06-07 02:12:52	2008-10-06	2008-10-06 22:47:00
`cp --preserve`	2025-06-07 02:12:52	2025-06-07	2025-06-07 02:12:52
`cp --preserve` with `umount`	2025-06-07 02:12:52	2008-10-06	2008-10-06 22:47:00
`cp --preserve` with Deleted File with `umount`	2025-06-07 05:18:42[a]	2008-10-06	2008-10-06 22:47:00
Time Travel `touch`	2025-06-07 02:12:52	2008-10-06	2008-10-06 22:47:00
Time Travel with mv	2008-10-06 22:47:00	2025-06-07[b]	2025-06-07 02:12:52[b]
Time Travel with `mv` and `touch`	2008-10-06 22:47:00	2008-10-06	2008-10-06 22:47:00

Table 2: Observed Timestamps for the FAT32 File System within FTK Imager for the targeted control file under different timestomping methods.
[a] The time of execution of the timestomp test [b] The timestamp of the access date and the modified timestamp of the original file; unchanged

4.1.2. Ext3 and Ext 4

Ext3 and Ext4 file systems are similar in structure, with Ext4 evolving from Ext3. During this research, one of the primary differences between a standard Ext3 file system and an Ext4 file system is the support for 64-bit addressing and the prevalence of additional checksums in the Ext4 file system structures. Analysis of the timestomping activity on Ext3 and Ext4 file systems shows that both file systems behave similarly. Therefore, the analysis results are presented as a single result set in Table 4, with caveats between the two file systems noted in the analysis as required. The table shows the timestamps for the created, accessed, modified, and change times for the file under different timestomping methods. The timestamps are shown in both human-readable format and referenced in encoded values.

touch Command

The filesystems had the same issues as FAT32, whereby the timestamps were not immediately updated in the operating system due to the file system cache. The `umount` command ensured the changes were reflected in the file system metadata, successfully updating the timestamps as expected.

Fundamental analysis of the file system metadata identified some superblock internal structures related to timestamps and checksums. Between Ext3/Ext4, the first 336 bytes of the superblock are mostly the same[6]. Ext4 uses two sectors of 512 bytes for the superblock, and while this is the same reserved space as for Ext3, many of the fields after the first 336 bytes are not used. Details of the inode structure are from the kernel.org guide[7].

[6] https://docs.kernel.org/filesystems/ext4/globals.html

[7] https://www.kernel.org/doc/html/latest/filesystems/ext4/inodes.html

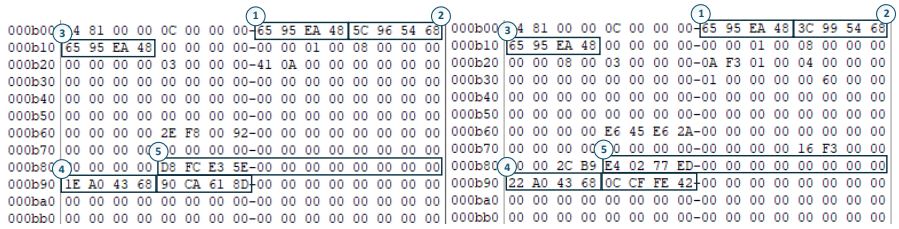


Figure 2: Ext3 (left) and Ext4 (right) File System Record Hex-Dump side-by-side of a stomped file with the following fields highlighted: 1. `atime` 2. `ctime`, 3. mtime, 4. `crtime` followed by its sub-second precision, 5. sub-second precision for `ctime`, `mtime`, and `atime`.

The journal examination shows valuable information that can be used to identify the timestomping activity. However, the journal is a relatively basic binary structure that is not easily readable. The basic principle of the journal is to store pending changes and record their successful commitment to the file system. The key journal entries related to timestomping are the Descriptor Block and Commit Block entries. Descriptor blocks contain information about the changes that are being made in the form of tags, storing the block number that is being written, along with other details about the journal entry[8]. Detailing the composition of the descriptor block is beyond the scope of this research and relates to details stored in the superblock of the journal. When a target file is known, it is relatively simple to identify the block of the directory involved and any changes. It is more complex to take a block, map it to a potential directory entry, and determine if there are changes of interest to timestamps. Figure 3 shows a journal record that relates to the timestomped file. The journal record stores the block number that is being written—in this instance, it is 00 00 00 2E, which is block 46—with the data following the description block being the actual data blocks to be written.

[8] https://docs.kernel.org/filesystems/ext4/globals.html

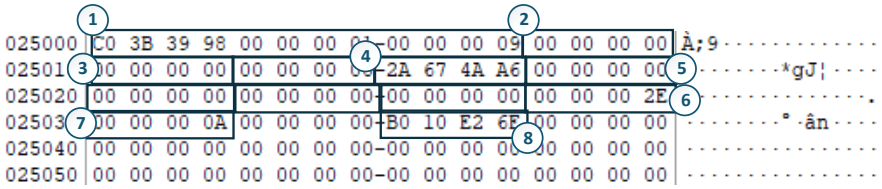


Figure 3: Descriptor Block Header with the magic number (0xC03B3998) block type and sequence number, 2. Tag representing the first block (00 00 00 00), 3. Flags for the first tag (none), 4. the checksum for the first tag, 5. a 16-byte UUID for the tag. 6. The tag.

cp Command

The results of using the `cp --preserve` command on the Ext3 and Ext4 file systems were similar to those of the FAT32 file system. The timestamps were not immediately updated in the operating system due to the file system cache, and a subsequent `umount` command was required to ensure that the changes were committed to the file system metadata. The `cp --preserve` command successfully preserved the original timestamps of the file; however, as with previous results, the creation timestamp was not changed when the file existed. The access and modified timestamps were updated to the current time, and the change timestamp was also updated to the current time.

Time Travel

The kernel cache was updated as expected in tests performed with time travel. In cases where no new files were created, i.e., no use of the mv command, all timestamps except the creation date were changed in the kernel cache. As expected, the creation date was not changed, as the file already existed. An interesting observation of these tests was that no changes were made to the file systems under time-travel scenarios, with a diff of the raw disk image showing only cursory changes to header information, such as the last mount time and the last write time of the file system. An example output of the stat command on the file /mnt/ext3/3DUfw.bin after the time-travel timestomping methods were executed is shown in Listing 3. The output shows that the access, modified, and

change times were updated to the time of the timestomping operation, while the birth time was kept at the original creation date.

```
Executing command: stat    /mnt/ext3/3DUfw.bin
File: /mnt/ext3/3DUfw.bin
Size: 12 Blocks: 8 IO Block: 4096 regular file
Device: 8,33 Inode: 12 Links: 1
Access: (0644/-rw-r --r --) Uid: (0/root) Gid: (0/root)
Access:    2008-10-06 22:47:01.019621999    +0000
Modify:    2008-10-06 22:47:01.019621999    +0000
Change:    2008-10-06 22:47:01.019621999    +0000
Birth:     2025-06-07 02:12:46.592999076    +0000
```

Listing 3: Output of executing the stat command on the file `/mnt/ext3/3DUfw.bin` after executing the time travel timestomping touch methods.

The output of the kernel cache is seen to be different from the raw disk image, shown in FTK Imager in Figure 4, which shows that the metadata of the access, modified, and change times was not updated on the raw disk. Raw data on the disk can be seen in Figure 4.

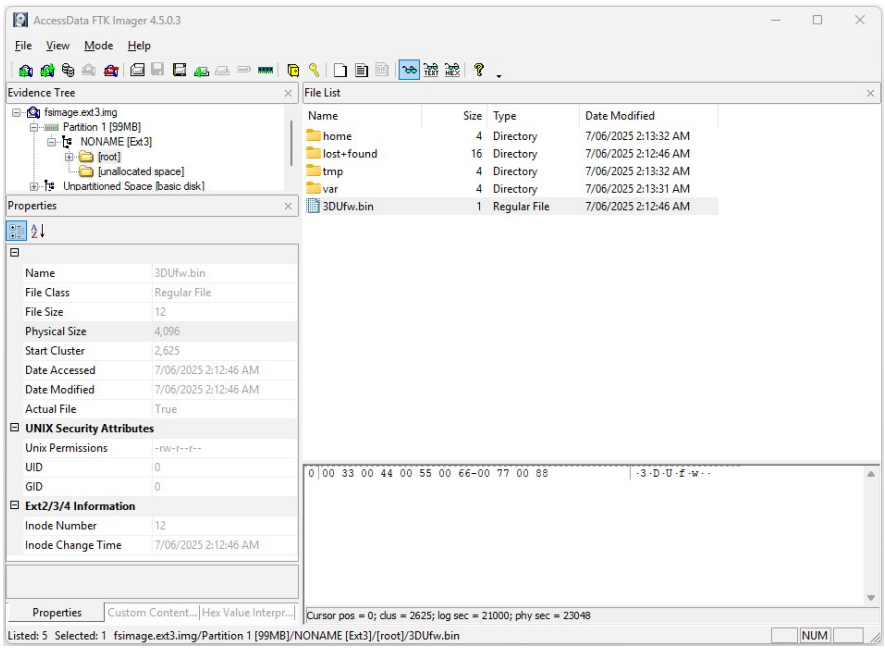


Figure 4: FTK Imager showing the metadata of the file `/mnt/ext3/3DUfw.bin` after the time-travel timestomping methods were executed. The metadata shows that the access, modified, and change times were not updated on the raw disk.

Debugfs

Tests using the debugfs command to modify the file system metadata successfully updated the file's timestamps. The `debugfs` command was used to change the files accessed, modified, created, and changed timestamps. As expected from previous research by Pomeranz (2024), the `debugfs` command did not update the kernel cache, and therefore, the timestamps were not updated in the operating system.

While the `debugfs` command could change the file's timestamps, it failed to change the modified timestamp of the file; this was unexpected. However, a file with a modified timestamp after its creation time is common on file systems and is not as anomalous as a file whose modified timestamp pre-dates its creation time.

Even though Ext3 and Ext4 file systems have a journal that records changes to the file system (Sirivaram & Roy, 2024), in the tests run with the `debugfs` command, no journal entries related to the timestomping activity were found.

4.2. Indirect Artifacts

To assess the indirect artifacts left by the timestomping methods, methods were grouped based on their perceived impacts, and one of the operating systems' drive images for each group was examined. To that extent, the drive images that were examined were for the following timestomp methods and file systems:

- FAT32 file system with the `cp --preserve` with a `umount` command
- Ext3 file system with the time-travel method utilizing the mv command
- Ext4 file system with the `debugfs` timestomp method

With these file systems selected, the drive of the operating system of the base image was mounted, and a hash of every file was taken. This was then repeated on each operating system drive of the selected methods. Finally, three scans of the hash list were performed: one to identify where files had been changed, another to identify where new files had been created in the selected operating system drives, and one to identify where files were removed. This reduced the analysis to a small, finite set of files modified from the base image.

It should be noted that the method of achieving these timestomp operations via SSH was done so that the `.bash_history` file was not written to, as the SSH session was disconnected before the shell exited. In any instance where the `.bash_history` is written, it will indicate the executed commands if the file has not been cleared or deleted.

The same set of files was identified as being modified in all analysis instances. These files were:

- `/var/log/wtmp` - the `wtmp` log file that contains a record of all logins and logouts, including SSH logins and commands executed.

- `/var/log/journal/874ba1681f1540d699d063320830a35c/system.journal` - the system journal file that records system events.
- `/var/lib/systemd/random-seed` - the random seed file used to seed the random number generator, which is used to generate random numbers for various system operations. This file was ignored as it was expected to change on every boot and not be changed in relation to the activity that had occurred.
- `/var/log/journal/874ba1681f1540d699d063320830a35c/system@24887703d2ba491abe5df7cde875392b-0000000000003914-000636f1f4cb2228.journal` - the system journal file that contains a record of the events for the boot process.

4.2.1. wtmp

The `wtmp` file is a binary file that contains a record of the logins and logouts of the system. While the contents of this file did not show any indicators of timestomping activity, it did indicate that the user had logged in.

```
        Sat Jun    7    02:18:19   2025 Term        tty1
        Sat Jun    21   01:21:12   2025  Init       tty1
        Sat Jun    21   01:21:12   2025  Login LOGIN tty1
```

Listing 4: The output of the last command on the `wtmp` file shows the login time of the user.

4.2.2. System Journal

It was expected that the journal files would contain a record of time-travel activity, and while there were some odd indicators, no entries were found in the journal that indicated the time had changed. However, the metadata of a new system journal file, shown in Figure 5, indicated the time of the timestomping activity. This file only appeared in test instances where time travel was used.

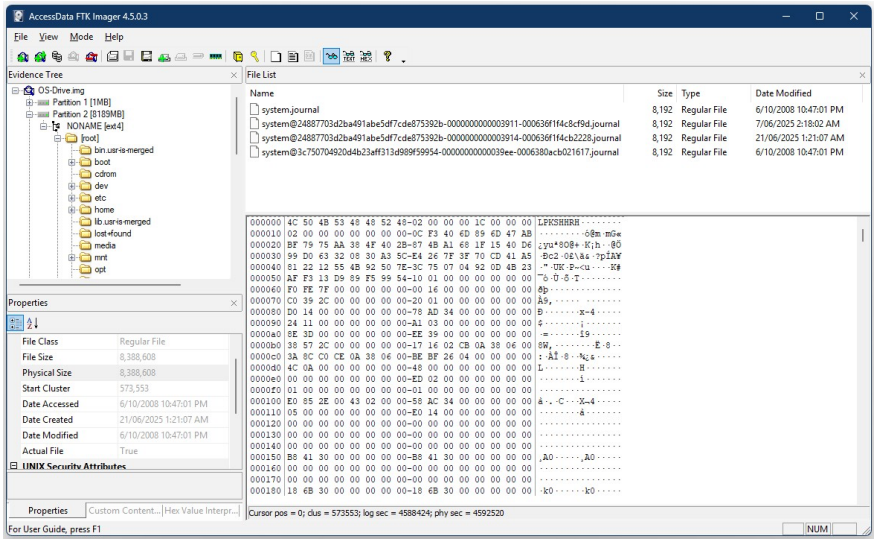


Figure 5: The modified system journal file, showing the time of the head and tail entries in the journal. The file is a binary file that contains a record of all system events, including timestomping activity. The file is shown in FTK Imager, with the modified t

Knowing this file existed with a modified timestamp, an in-depth examination of the binary structures of this file was conducted, examining the journal header for key markers of the machine ID, boot ID, and the associated timestamps of the head (first) and tail (last) entries in the journal, with a sample output in Listing 5. The details of the journal file format were taken from the freedesktop.org website[9]. This data was also cross-referenced with the output of the `journalctl` command, specifically utilizing the JSON output format to get as much information as possible about data in the underlying journal files.

```
"Signature": "LPKSHHRH",
"CompatibleFlags":2,
"IncompatibleFlags":28,
```

[9] https://www.freedesktop.org/wiki/Software/systemd/journal-files/

```
"State":2,
"Reserved": "AAAAAAAAAA==",
"FileId": "DPNAbYltR6u/eXWqOE9AKw==",
"MachineId": "h0uhaB8VQNaZ0GMyCDCjXA==",
"MachineId_Hex": "874ba1681f1540d699d063320830a35c",
"BootId": "5CZ/P3DNQaWBIhJVS5JQfg==",
"BootId_Hex: "e4267f3f70cd41a5812212554b92507e",
"SequenceNumberId": "PHUHBJINSyOv8xPZifWZVA==",
"SequenceNumberId_Hex": "3c750704920d4b23aff313d989f59954",
"HeaderSize": 272,
"ArenaSize": 8388336,
"HeadEntryRealtime": 1750468867003927,
"TailEntryRealtime": 1750468929817658,
"TailEntryMonotonic": 69648318
```


Listing 5: The output of parsing the journal header of the system journal file shows key metadata such as the machine ID, boot ID, and the time of the head and tail entries in the journal. The output is taken from a JSON-formatted object for straightforward interpretation.

Along with the anomalous journal file in the timeframe of the targeted time-travel time, the journal recorded details about disabling Network Time Protocol (NTP) on the host. An excerpt of the `journalctl` command is shown in Listing 6, which shows the time-travel activity and the disabling of NTP on the host.

```
[system] Successfully activated service 'org.freedesktop.timedate1'
Started systemd-timedated.service - Time & Date Service.
dated[876]: systemd-timesyncd.service: Disabling unit.
Reloading...
Reloading finished in 489 ms.
Stopping systemd-timesyncd.service - Network Time Synchronization...
dated[876]: Set NTP to disabled.
systemd-timesyncd.service: Deactivated successfully.
Stopped systemd-timesyncd.service - Network Time Synchronization.
```

Listing 6: The output of the journalctl command shows the time-travel activity and the disabling of NTP on the host.

Initial analysis of the system journal concerning the debugfs command and tests involving the use of the `umount` command did not show any related entries. As this was unexpected, a further test run of the `debugfs` command was performed, and the journal was examined on a live system. The hypothesis is that the system journal was cached, and the short time between when the activity was performed and when the system was suspended for preservation was not enough time for the journal to be updated on disk.

A subsequent execution of the test environment, focusing on both `debugfs` and `cp --preserve` with `umount`, was performed to allow for the analysis of a live system journal. This interactive live analysis of test runs showed no entries that identified the use of the `debugfs` or `umount` commands being executed.

5. Discussion

5.1. Ext3/Ext 4

While it was generally possible to change the accessed and modified timestamps with little issue, the existence of the `debugfs` command allowed for the change of the creation and change timestamps on Ext3 and Ext4 file systems; however, as identified in Pomeranz (2024), ensuring these times are synced in the kernel is more involved and requires that the metadata be reloaded.

Regarding identifying the most basic method of timestomping, using the touch command or the `cp --preserve` command, the results indicate that it is impossible to timestomp the creation and change timestamps on Ext3 and Ext4 file systems.

5.2. File System Journal Observations

The information in the Ext3/Ext4 file system journal helps identify whether a file has been timestomped; however, there are limitations in knowing the block of the metadata suspected of being timestomped. The journal operates at a block level, making applying its records at a file metadata level challenging. While this is not entirely impossible, it is not practical to do so in the scope of this research.

Author Name, email@address

Additionally, when examining the results of time travel, it was found that the journal did not contain any entries related to the timestomping activity, and that no changes were made to the raw disk. The reason for this is unknown; however, it is hypothesized that some form of protection mechanism is in place that prevents what is deemed outdated transactions, such as those before the last transaction in the journal, from being performed on the file system.

5.3. System Journal Observations

Examining the system journal, it was found that there were no entries related to the timestomping activity even when using commands such as `debugfs` or `umount`. This is unexpected, as it was hypothesized that the system journal would contain entries related to direct access to the disk, in the case of `debugfs`, or the unmounting of the file system, as is the case with `umount`.

For time travel, the artifacts in the system journal were also minimal, with only the disabling of the NTP service being recorded. There was, however, the presence of a new system journal file created at the time of the target time-travel time. An examination of this file showed that its contents were out of place with respect to the timestamp of the file. So, while the system journal did not directly contain entries related to the timestomping activity, there was a significant anomaly in the presence of a new system journal file, which, in this case, dated back 17 years.

5.4. Shell Environment Effects

Shell history files can play an important role in investigating malicious activity. However, these files are not always correct, time-stamped, or present. This research utilized a bash shell through an SSH connection to the virtual machine. The shell history file, typically `.bash_history` in the user's home directory, was avoided in the testing by disconnecting the SSH session and terminating the shell before writing the history file; some methods can change this behavior[10]. Recording actions performed in a shell

[10] https://www.cherryservers.com/blog/a-complete-guide-to-linux-bash-history

environment is again a function of the EDR system, further highlighting the importance of EDR systems in detecting and responding to potential adversarial activity.

5.5. Destructive File Movements

Many of the timestomping methods tested in this research involved moving files to and from different locations on the file system. While this can be an effective way to change timestamps, it may also cause problems with file locking from other programs that may prevent the action from occurring. While this may not be an issue for binaries that the adversary has dropped, this becomes more complicated when the adversary is attempting to timestomp files that are in use by other programs, such as the `/etc/passwd` or `/etc/shadow` files, to disguise adding a new user.

5.6. User Access Levels

The privilege a user has in an environment has a significant impact on their ability to perform low-level operating system functions. While methods of bypassing checks and escalating privileges exist, these methods often leave indicators of their own, often in the form of additional artifacts written to hard disks Lin et al., 2022. Further, it is the function of EDR (Endpoint Detection and Response) systems to monitor these types of activities, which would provide a layer of early detection of potential adversarial activity.

The testing environment in this research was designed to simulate a highly privileged user, the root user, to ensure that the full range of timestomping methods could be tested. However, it is recognized that unprivileged users will not have the same access to low-level file system operations, such as those provided by debugfs or raw disk writing. This limitation may prevent them from performing specific timestomping techniques, such as modifying file system journals or directly manipulating inode metadata. It is, therefore, important that organizations consider the access levels of users as a mitigation against timestomping techniques.

5.7. Detection and Mitigation

While detection and mitigation strategies were not the focus of this research, some methods to detect the presence of timestomping activity can be drawn from the results:

- Ensure that investigators are aware of the potential access and modified dates of files to be easily changed. It is important to understand that alterations to the changed and creation timestamps of files are far more difficult, require higher privileges and bespoke tools, and will often leave other artifacts related to privilege escalation prior.
- Deploy EDR systems to monitor low-level file system operations, such as those performed by `debugfs` or raw disk writing. These systems can provide early detection of potential adversarial activity and alert security teams to investigate further.

In terms of mitigations for timestomping activity, the following strategies are recommended:

- Ensure that users are assigned under the least privilege principle and do not have access to low-level file system operations.
- Where possible, remove tools allowing low-level file system operations, such as debugfs.

5.8. Summary of Artifacts

A table of timestomping methods and artifacts that can be used to detect them has been summarized in Table 5, in the Appendix. This table provides a quick reference for investigators to identify potential timestomping techniques and the associated artifacts that may be present in the system.

6. Implications for Future Research

Future research should explore testing in more realistic and larger-scale environments where legitimate file system activity occurs alongside timestomping. This would help assess how effectively timestomped files can be identified amidst typical system usage. Additionally, further investigation is needed to determine how long it takes for journal entries to age out, thereby limiting their usefulness in detecting timestomping activity.

7. Conclusion

This research presents a comprehensive examination and unified reference of timestomping methods. An experimental methodology was developed to systematically analyze direct and indirect forensic artifacts across FAT32, Ext3, and Ext4 file systems. The results demonstrate that while specific timestomping techniques can successfully alter file timestamps, they often leave detectable traces within file system metadata or at the system level, such as logs or audit trails. The findings highlight the importance of correlating artifacts with specific adversary capabilities and access levels. This research provides a practical framework for testing timestomping methods and analyzing direct artifacts in the filesystem and indirect artifacts that may be found in locations relating to Operating System logs. This comprehensive detail of the artifacts from timestomping methods supports a more effective detection and response in Linux environments where the use of timestomping is suspected.

Author Name, email@address

References

ATT&CK, M. (2021). Indicator removal: Timestomp. *https://attack.mitre.org/versions/v16/techniques/T1070/006/*.

Bhat, W. A., AlZahrani, A., & Wani, M. A. (2021). Can computer forensic tools be trusted in digital investigations? *Science & Justice*, *61*(2), 198–203.

Chunduri, H., Anand, P. M., Shukla, S. K., & Sai Charan, P. V. (2024). Trusted yet disguised: Analysing the subversive role of lolbins in contemporary cyber threats. *2024 IEEE International Conference on Big Data (BigData)*, 2596–2605.

https://doi.org/10.1109/bigdata62323.2024.10825239 Forensic Focus. (2015). Linux timestamps, oh boy! https://www.forensicfocus.com/articles/linux-timestamps-oh-boy/

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley Longman Publishing Co., Inc.

Hosgor, E. C. (2018). *Detection and mitigation of anti-forensics using forensic tools* [Thesis].

Lin, Z., Wu, Y., & Xing, X. (2022). Dirtycred. https://doi.org/10.1145/3548606.3560585

Mohamed, A., & Khalid, C. (2019). Detection of timestamps tampering in ntfs using machine learning. *Procedia Computer Science*, *160*, 778–784. https://doi.org/https://doi.org/10.1016/j.procs.2019.11.011

Olsson, J., & Boldt, M. (2009). Computer forensic timeline visualization tool. *digital investigation*, *6*, S78–S87.

Pomeranz, H. (2024). More on ext4 timestamps and timestomping. https://righteousit.com/2024/09/04/more-on-ext4-timestamps-and-timestomping/

Singh, B., & Gupta, G. (2019). Analyzing windows subsystem for linux metadata to detect timestamp forgery. *Advances in Digital Forensics XV: 15th IFIP WG 11.9 International Conference, Orlando, FL, USA, January 28–29, 2019, Revised Selected Papers 15*, 159–182.

Author Name, email@address

Sirivaram, S. B., & Roy, S. (2024). Deleted file recovery for the linux file system (ext4): Finding the state-of-the-art. *2024 12th International Symposium on Digital Forensics and Security (ISDFS)*, 1–6.

Strom, B. E., Applebaum, A., Miller, D. P., Nickels, K. C., Pennington, A. G., & Thomas, C. B. (2018). MITRE ATT&CK: Design and philosophy. In *Technical report*. The MITRE Corporation.

Thierry, A., & Müller, T. (2022). A systematic approach to understanding macb timestamps on unix-like systems. *Forensic Science International: Digital Investigation*, *40*, 301338. https://doi.org/https://doi.org/10.1016/j.fsidi.2022.301338

Yaacoub, J.-P. A., Noura, H. N., Salman, O., & Chehab, A. (2021). Digital forensics vs. anti-digital forensics: Techniques, limitations and recommendations. *arXiv preprint arXiv:2103.17028*. https://doi.org/10.48550/arxiv.2103.17028

Author Name, email@address

Appendix

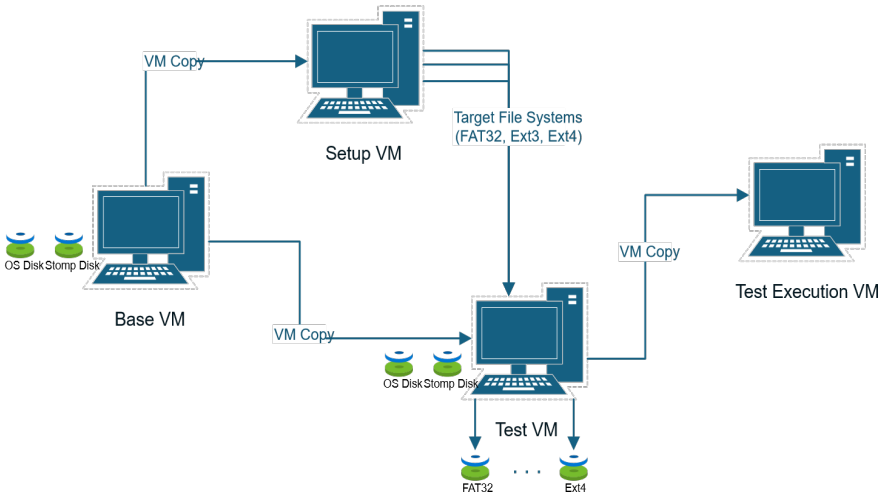


Figure 6: Virtual Machine Relationships

Phase	Steps
Setup	1. Create a minimal, standard installation of Ubuntu 24.02. 2. Create a set of file systems of the specified type (FAT32, Ext3, or Ext4). 3. Generate a test file with identifiable data and file metadata that considers both UTF-8 and UTF-16 encoding on each of the file systems. 4. Hash the control file placed on the file system to ensure its correctness.
Execution	1. Attach the file system to a virtual machine running a Linux operating system and boot the environment. 2. Connect to the virtual machine via SSH and execute the timestomping method on the control file. 3. For analysis, suspend the system and preserve a copy of the virtual machine state, including all attached disks. 4. Resume the virtual machine and record the results of basic live analysis commands on the operating system to determine whether the operation was successful before shutting down and discarding the virtual machine.
Analyse	**Direct Artifact Analysis** 1. Acquire a hex-dump of the initial and modified state of the file system for comparison. 2. Examine the difference between the two to identify sectors with differences in data. 3. Identify the relevance of the sectors to file system metadata. 4. Record artifacts that are related to timestomping.

	Indirect Artifact Analysis 1. Acquire a hex dump of the initial and modified state of the Operating System disk for comparison. 2. Examine the differences between the two Operating System drive states to identify where there are differences in data. 3. Identify the relevance of the sectors to operating system artifacts. 4. Record artifacts that are related to timestomping.

Table 3: Artifact Testing: Setup and Execution Steps

Ext3/Ext4	Target Date/Time: 2008-10-06 22:47:00 Encoded Timestamp: 0x6595EA48			
Method	**Created Timestamp**	**Accessed Timestamp**	**Modified Timestamp**	**Change Timestamp**
`touch`	2025-06-07 02:12:52	2025-06-07 02:12:52	2025-06-07 02:12:52	2025-06-07 02:12:52
`touch` with `umount`	2025-06-07 02:12:52	2008-10-06 22:47:00	2008-10-06 22:47:00	2025-06-20 08:59:40[a]
`cp --preserve` with `umount`	2025-06-07 02:12:50	2008-10-06 22:47:01	2008-10-06 22:47:01	2025-06-21 12:47:15[a]
`cp --preserve` with Deleted File with `umount`	2025-06-21 13:07:46[a]	2008-10-06 22:47:01	2008-10-06 22:47:01	2025-06-21 13:07:46[a]
Time Travel `touch`	2025-06-07 02:12:50	2025-06-07 02:12:50	2025-06-07 02:12:50	2025-06-07 02:12:50
Time Travel with `mv`	2025-06-07 02:12:50	2025-06-07 02:12:50	2025-06-07 02:12:50	2025-06-07 02:12:50
Time Travel with `mv` and `touch`	2025-06-07 02:12:50	2025-06-07 02:12:50	2025-06-07 02:12:50	2025-06-07 02:12:50
`debugfs`	2008-10-06 22:47:01	2008-10-06 22:47:01	2025-06-07 02:12:50	2008-10-06 22:47:01

Table 4: Observed Timestamps for Ext3/Ext4 File System within FTK Imager for the targeted control file under different timestomping methods.

[a] The time of execution of the timestomp test, [b] The timestamp of the access date and the modified timestamp of the original file, unchanged

Method	Direct Artifacts	Indirect Artifacts
`touch`	Modified timestamps prior to creation or change timestamps	None
`debugfs`	A mismatch between timestamps from commands in the Operating System versus timestamps extracted from the raw disk	None
time-travel	None. It may not be successful, depending on the file system.	Evidence of changes to the NTP status. Odd system journal files that are not in relevant time periods
`cp --preserve`	Modified timestamps prior to creation or change timestamps	None

Table 5: Summary of Timestomping Methods and Potential Artifacts (All cells are exploratory)